CHƯƠNG 3: XỬ LÝ BẤT ĐỒNG BỘ TRONG NODEJS

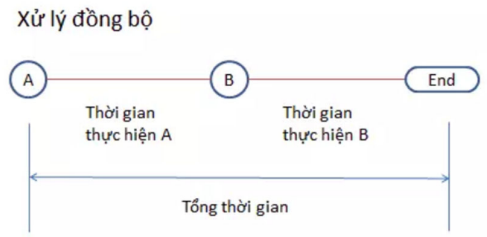
Mục tiêu chương 03: Chương này sẽ giới thiệu đến các bạn về xử lý bất đồng bộ trong Nodejs, tìm hiểu về các kỹ thuật Callback, Promise, Async await, … để xử lý bất đồng bộ trong Nodejs.

3.1. GIỚI THIỆU VỀ XỬ LÝ BẤT ĐỒNG BỘ

Trước khi vào vấn đề chúng ta cùng tìm hiểu xem đồng bộ (**Synchronous**) và bất đồng bộ (**Asynchronous**) là gì?

Đồng bộ (**Synchronous**) **:** Hiểu đơn giản thì đồng bộ có nghĩa là thực hiện các công việc một cách tuần tự, công việc này xong thì mới được thực hiện các công việc khác. Ví dụ có 2 công việc A và B thì khi có nghĩa là A thực hiện xong trước rồi mới tới lượt B. Như vậy tổng thời gian hoàn thành sẽ bằng tổng thời gian 2 công việc.

Điều này nó sẽ ảnh hưởng đến hiệu suất của người dùng. Giả sử một **request** gửi lên **server** yêu cầu server thực hiện chức năng như import file hoặc đọc ghi file thì lúc này server sẽ mất nhiều thời gian để xử lý những việc này. Điều đó cũng đồng nghĩa với việc trong lúc server thực hiện chức năng đó thì sẽ không thể thực hiện thêm một hành động nào khác. Chưa kể trong lúc server đang thực hiện những việc đó thì lại có thể có nhiều request được gởi đến trong khi server chưa thực hiện xong, điều này có thể gây **crash server** (treo).

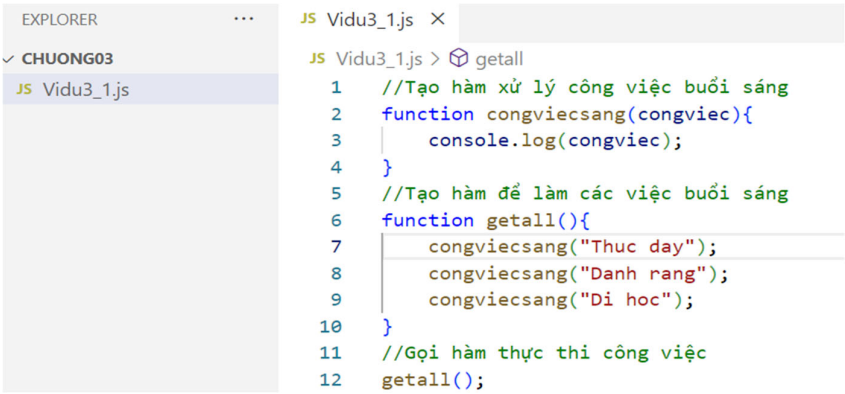


Hình 3.1: Xử lý đồng bộ

Ví dụ 3.1: Ví dụ về đồng bộ

- Tạo thư mục **Chuong03** trong thư mục **D:\THNodejs** đã có:

- Tạo tập tin **Vidu3_1.js** trong thư mục **Chuong03**, viết Code như bên dưới:



- Biên dịch chương trình:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\BTTH_Nodejs_Khuong\Chuong03> node Vidu3_1
Thuc day
Danh rang
Di hoc
```

- Như vậy ta thấy các công việc sẽ thực hiện theo thứ tự tuần tự, hết dòng lệnh này đến dòng lệnh khác.

Ngược lại với đồng bộ, **bất đồng bộ** (**Asynchronous**) là việc các sự kiện, công việc có thể được thực hiện một cách độc lập, hoạt động song song không gây ảnh hưởng tới luồng công việc chính. Điều này sẽ tối ưu được thời gian chờ, qua đó làm giảm tổng thời gian cần để hoàn thành một nhiệm vụ.

Nhưng kết quả của công việc thực hiện sau có thể được trả về trước kết quả của công việc chạy trước nó, do vậy kết quả trả về cuối cùng có thể sẽ không theo thứ tự đã định sẵn, yêu cầu chúng ta cần kiểm soát chặt chẽ hơn.

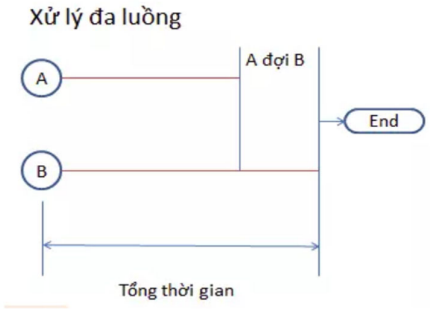
Ví dụ 3.2: Ví dụ về bất đồng bộ

- Tạo tập tin **Vidu3_2.js** trong thư mục **Chuong03**, viết Code như bên dưới và biên dịch chương trình.

```
EXPLORER
CHUONG03
JS Vidu3_1.js
JS Vidu3_2.js

JS Vidu3_2.js
JS Vidu3_2.js > getall
1   //Tạo hàm xử lý công việc buổi sáng
2   //setTimeout thiết lập độ trễ xử lý hàm ngẫu nhiên 1-99
3   function congviecsang(congviec){
4       setTimeout( () => {
5           console.log(congviec);
6       }, Math.random()*100)
7   }
8
9   //Tạo hàm để làm các việc buổi sáng
10  function getall(){
11      congviecsang("Thuc day");
12      congviecsang("Danh rang");
13      congviecsang("Di hoc");
14  }
15  //Gọi hàm thực thi công việc
16  getall();
17

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\BTTH_Nodejs_Khuong\Chuong03> node Vidu3_2
Di hoc
Danh rang
Thuc day
PS D:\BTTH_Nodejs_Khuong\Chuong03>
```

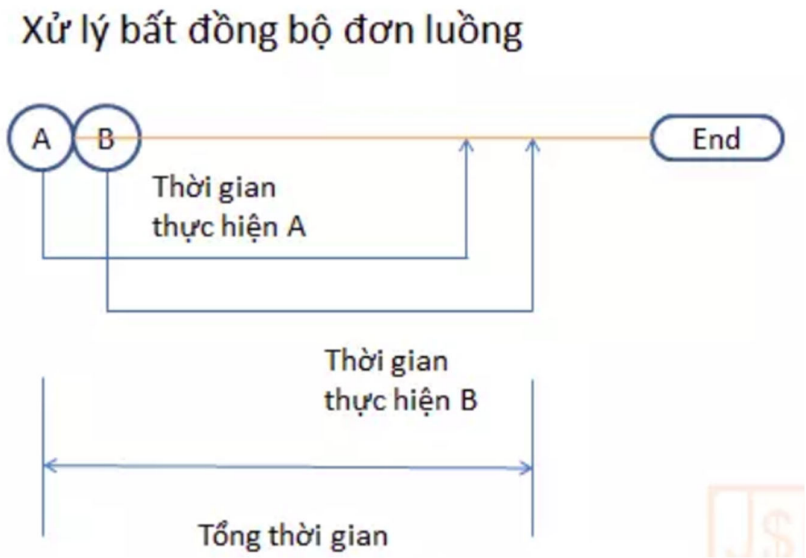
Mặc dù những hàm trên được xếp theo thứ tự công việc, nhưng chúng lại độc lập với nhau do setTimeout()), do vậy những hàm này là bất đồng bộ. Các **congviec** sẽ được lấy ra một cách ngẫu nhiên không theo thứ tự.

Multi-thread: Để khắc phục tình trạng này, các ngôn ngữ lập trình như C/C++, Java,... sẽ sử dụng cơ chế đa luồng (**multi-thread**). Nghĩa là mỗi công việc tốn thời gian sẽ được thực hiện trên một thread riêng biệt mà không can thiệp vào thread chính. Bạn vẫn có thể thực hiện các công việc tốn thời gian mà vẫn có thể bắt các sự kiện ở thread chính.



Hình 3.2: Xử lý đa luồng

Javascript và cơ chế bất đồng bộ: Đối với **javascript** thì nó là ngôn ngữ **Single thread**, tức là nó chỉ có 1 thread duy nhất, nên các bạn không thể sử dụng multi thread như những ngôn ngữ khác mà phải sử dụng **cơ chế xử lý bất đồng bộ**.



Hình 3.3: Xử lý bất đồng bộ đơn luồng

Với cách xử lý bất đồng bộ, khi A bắt đầu thực hiện, chương trình tiếp tục thực hiện B mà không đợi A kết thúc. Việc mà bạn cần làm ở đây là cung cấp một cơ chế để chương trình thực hiện đúng theo trình tự yêu cầu.

Cơ chế giúp bạn thực hiện việc này trong JavaScript có thể là sử dụng **Callback**, **Promise** hoặc **Async/await**.

3.2. CALLBACK

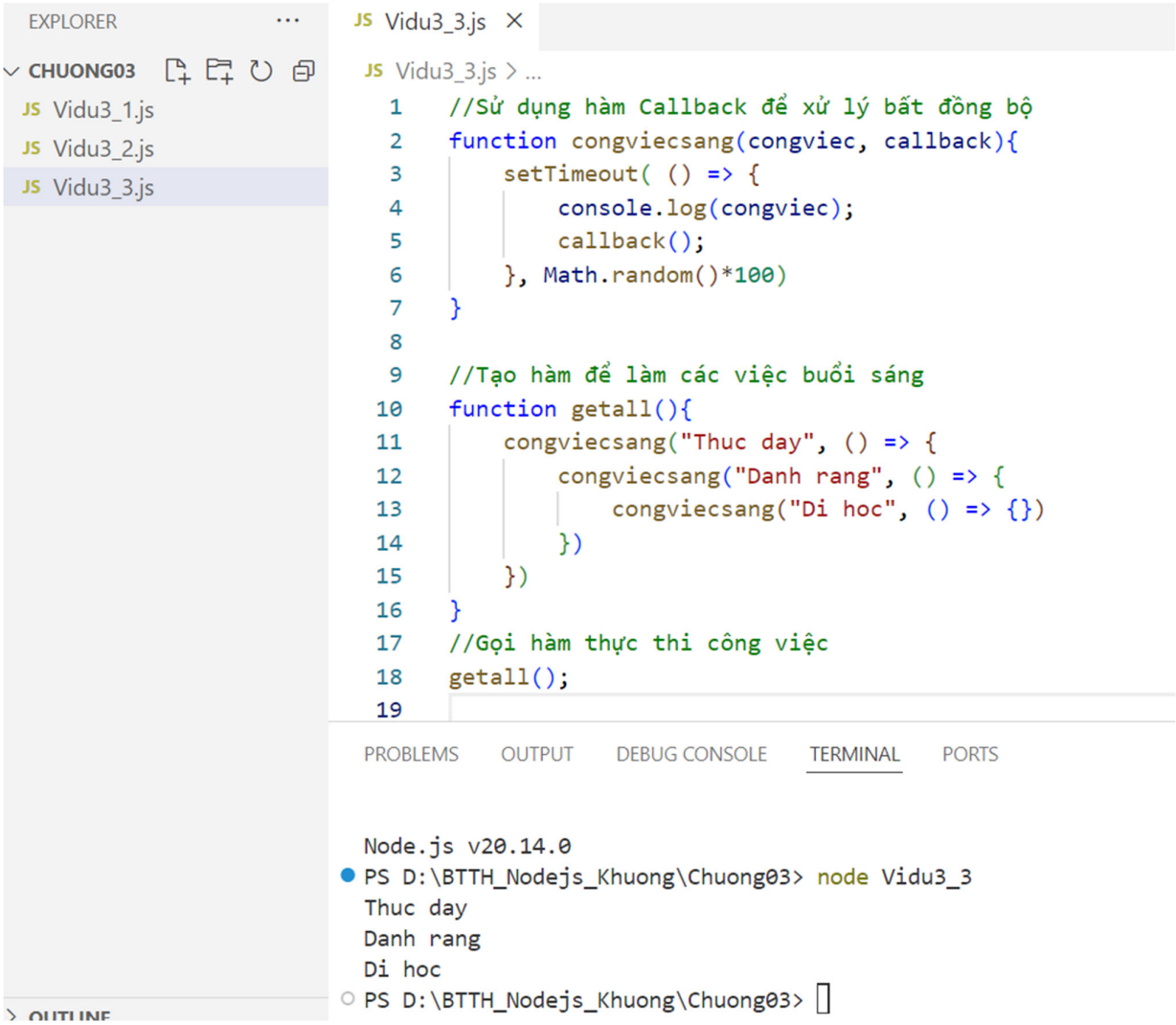
3.2.1. Giới thiệu về Callback

Callback là hàm được truyền vào một hàm khác (tạm gọi là hàm cha) giống như **một tham số** của hàm đó. Khi hàm cha thực hiện xong, thường thì hàm này sẽ tốn nhiều thời gian hoàn thành, hàm callback sẽ được thực hiện. Đây là cách dễ thực hiện nhất dùng để xử lý bất đồng bộ trong **JS**.

3.2.2. Ví dụ về callback

Ví dụ 3.3: Sử dụng hàm callback để xử lý bất đồng bộ trong Nodejs:

- Tạo tập tin **Vidu3_3.js** trong thư mục **Chuong03**, viết Code như bên dưới và biên dịch:



- Như vậy hàm **callback** đã được truyền vào hàm **congviecsang** như là một tham số, đã xử lý việc bất đồng bộ để các công việc thực hiện theo đúng trình tự yêu cầu khi biên dịch.

Ghi chú về cách viết hàm trong JavaScripts:

```
function() {}      function(a) {}      function(a,b) {}
```

viết ngắn gọn thành

```
() => {}           a => {}             (a,b) => {}
```

Nhưng có một vấn đề phát sinh ở đây, khi chúng ta muốn nhiều hàm bất đồng bộ thực hiện theo đúng thứ tự tuần tự, chúng ta phải gọi nhiều hàm **callback**() lồng nhau, khi đó **code** của chúng ta nhìn sẽ phức tạp, không tối ưu và rất khó kiểm soát. Trường hợp này gọi **là Callback Hell**.

```
function register()
{
    if (!empty($_POST)) {
        $msg = '';
        if ($_POST['user_name']) {
            if ($_POST['user_password_new']) {
                if ($_POST['user_password_new'] === $_POST['user_password_repeat']) {
                    if (strlen($_POST['user_password_new']) > 5) {
                        if (strlen($_POST['user_name']) < 65 && strlen($_POST['user_name']) > 1) {
                            if (preg_match('/^[a-z\d]{2,64}$/i', $_POST['user_name'])) {
                                $user = read_user($_POST['user_name']);
                                if (!isset($user['user_name'])) {
                                    if ($_POST['user_email']) {
                                        if (strlen($_POST['user_email']) < 65) {
                                            if (filter_var($_POST['user_email'], FILTER_VALIDATE_EMAIL)) {
                                                create_user();
                                                $_SESSION['msg'] = 'You are now registered so please login';
                                                header('Location: ' . $_SERVER['PHP_SELF']);
                                                exit();
                                            } else $msg = 'You must provide a valid email address';
                                        } else $msg = 'Email must be less than 64 characters';
                                    } else $msg = 'Email cannot be empty';
                                } else $msg = 'Username already exists';
                            } else $msg = 'Usernamo must be only a-z, A-Z, 0-9';
                        } else $msg = 'Username must be between 2 and 64 characters';
                    } else $msg = 'Password must be at least 6 characters';
                } else $msg = 'Passwords do not match';
            } else $msg = 'Empty Password';
        } else $msg = 'Empty Username';
        $_SESSION['msg'] = $msg;
    }
    return register_form();
}
```

Hình 3.4: Callback hell

Để giải quyết vấn đề này, **Promise** ra đời, giúp các đoạn code xử lí bất đồng bộ trở nên sáng sủa và dễ dàng debug hơn.